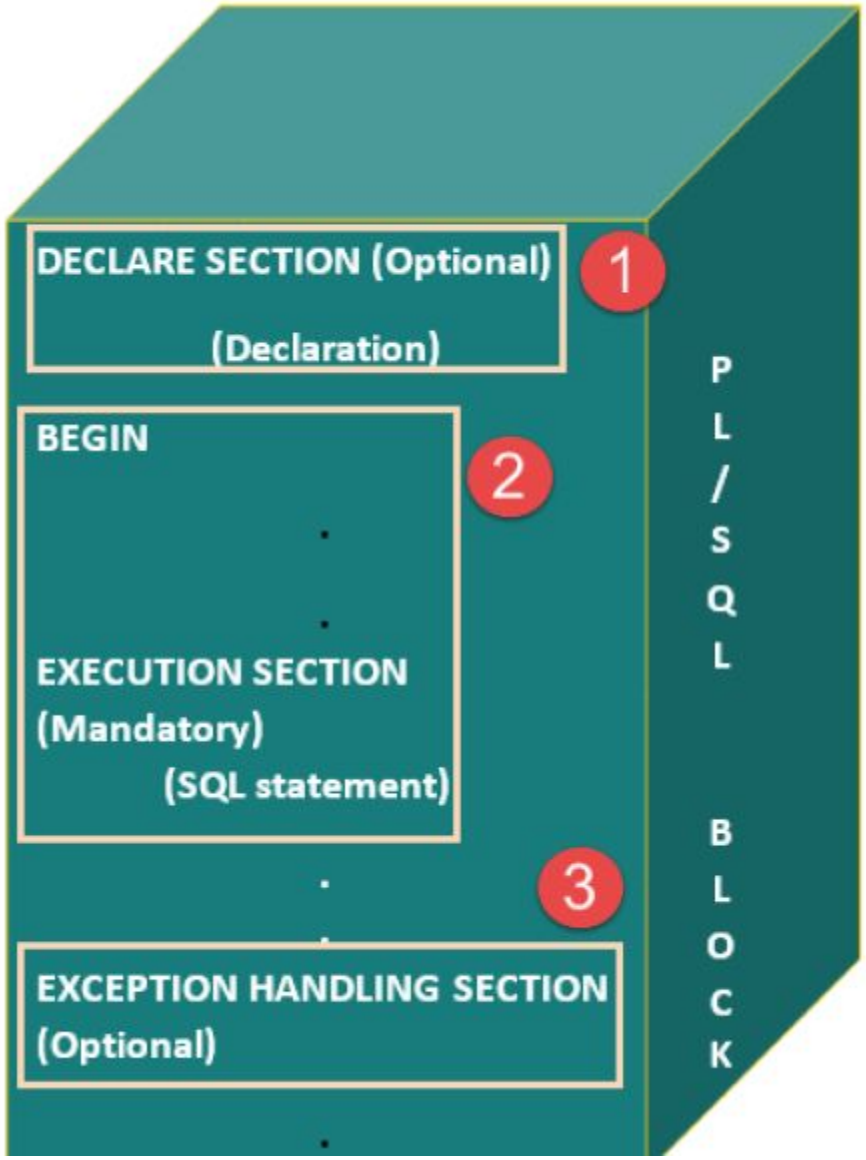
- PL/SQL Introduction
- •PL/SQL is a combination of SQL along with the procedural features of programming languages.
- •Basic Syntax of PL/SQL which is a **block-structured** language; this means that the PL/SQL programs are divided and written in logical blocks of code. Each block consists of three sub-parts
- •Every PL/SQL statement ends with a semicolon (;).
- • Following is the basic structure of a PL/SQL block −
-
- DECLARE <declarations section>
- BEGIN <executable command(s)>
- EXCEPTION <exception handling>
- END;

- Pl/SQL Block structure



Pl/SQL Block structure Explanation Sections	Description
Declarations	•This section starts with the keyword DECLARE. • It is an optional section and defines all variables, cursors, and other elements to be used in the program.
Executable Commands	•This section is enclosed between the keywords BEGIN and END and it is a mandatory section. •It consists of the executable PL/SQL statements of the program. •It should have at least one executable line of code.
Exception Handling	•This section starts with the keyword EXCEPTION. •This optional section contains exception(s) that handle errors in

- Stored Procedure- Parameters In MySQL, a parameter has one of three modes:

- IN OUT
- INOUT

Stored Procedure- Parameters

IN – is the default mode. When you define an IN parameter in a stored procedure, the calling program has to pass an argument to the stored procedure.

OUT – the value of an OUT parameter can be changed inside the stored procedure and its new value is passed back to the calling program

INOUT – an INOUT parameter is the combination of IN and OUT parameters. It means that the calling program may pass the argument, and the stored procedure can modify the INOUT parameter and pass the new value back to the calling program

- Assignment
- •Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory. Write a PL/SQL block of code for the following requirements:-
- •**Schema:**
- 1. Borrower(Roll_no, Name, DateofIssue, NameofBook, Status)
- 2. Fine(Roll_no,Date,Amt)
- •**Accept roll_no & name of book from user**.
- •Check the number of days (from date of issue), if days are between 15 to 30 then fine amount will be Rs 5per day.
- •If no. of days>30, per day fine will be Rs 50 per day & for days less than 30, Rs. 5 per day.
- •After submitting the book, status will change from I to R.
- •If condition of fine is true, then details will be stored into fine table.

- Assignment Required Functions- CURDATE()
- •The CURDATE() function returns the current date
- •.
- •**Note:** This function returns the current date as a "YYYY-MM-DD" format
- •**Example**
- •Return current date:
- •SELECT CURDATE();

- Assignment Required Functions- DATEDIFF()
- •The DATEDIFF() function returns the difference in days between two date values.
- •**Syntax**
- •DATEDIFF(*date1*, *date2*)
- •**Example**
- •Return the difference in days between two date values:
- •SELECT DATEDIFF("2017-06-25", "2017-06-15");

- Assignment Tables-before procure run
- **Roll_no Name DateofIssue NameofBook Status**
- 1 Amita 2017-06-25 Java I
- 2 Sonakshi 2017-07-10 Networking I
- 3 Nira 2017-05-22 MySQL I
- 4 Jagdish 2017-06-10 DBMS I
- 5 Jayashree 2017-07-05 MySQL I
- 6 Kiran 2017-06-30 Java I

- Fine Table **Roll_no Date Amt**

```
CREATE TABLE Assignment (
    Roll_no INT PRIMARY KEY,
    Name VARCHAR(50),
    DateofIssue DATE,
    NameofBook VARCHAR(50),
    Status CHAR(1)
);

INSERT INTO Assignment (Roll_no, Name, DateofIssue, NameofBook, Status)
VALUES
(1, 'Amita', '2017-06-25', 'Java', 'I'),
(2, 'Sonakshi', '2017-07-10', 'Networking', 'I'),
(3, 'Nira', '2017-05-22', 'MySQL', 'I'),
(4, 'Jagdish', '2017-06-10', 'DBMS', 'I'),
(5, 'Jayashree', '2017-07-05', 'MySQL', 'I'),
(6, 'Kiran', '2017-06-30', 'Java', 'I');
```

```
CREATE TABLE Fine (
    Roll_no INT,
    Date DATE,
    Amt DECIMAL(6,2),
    FOREIGN KEY (Roll_no) REFERENCES Assignment(Roll_no)
);
```

Stored Procedure Example in MySQL To find difference in current date and issue date

```
  delimiter $

Create procedure P1(
   In rno1 int(3),
   name1 varchar(30)
)
begin
   Declare i_date date;
   Declare diff int;

   select DateofIssue into i_date
   from stud
   where Rno = rno1 and NameofBook = name1;

   SELECT DATEDIFF(CURDATE(), i_date) into diff;
End;

delimiter ;
```

- Step-by-step Explanation
- 1. delimiter $
- Changes the default command delimiter from ; to $

- This is necessary inside stored procedures, where we use ; within the code, so we change the final delimiter to $.

- 2. Create procedure P1(...)
- You are creating a procedure named P1

- It takes two input parameters:

- rno1: the roll number (integer)

- name1: name of the book (string)

- 3. Declare i_date date;
- Declares a variable called i_date of type DATE

- It will be used to store the date when the book was issued

4. Declare diff int;

Declares a variable called diff of type INT

It will store the number of days between issue date and today

5. select DateofIssue into i_date ...
Looks in the stud table and fetches the DateofIssue where:

Rno = the roll number passed (rno1)

NameofBook = the book name passed (name1)

The result is saved into the variable i_date

6. SELECT DATEDIFF(CURDATE(), i_date) into diff;
CURDATE() gives today's date

DATEDIFF() calculates how many days have passed since the book was issued

The result is stored in diff

Stored Procedure Example in MySQL - To change status from I to R

```
delimiter $

Create procedure P2(
    In rno1 int(3),
    name1 varchar(30)
)
begin
    Declare i_date date;
    Declare diff int;

    select DateofIssue into i_date
    from stud
    where Rno = rno1 and NameofBook = name1;

    SELECT DATEDIFF(CURDATE(), i_date) into diff;
```

```
If diff > 15 then
        Update stud
        set status = 'R'
        where Rno = rno1 and NameofBook = name1;
    End if;
End;

$

delimiter ;

call p2(1, 'DBMS');
```

1. delimiter $

Changes the command delimiter from ; to $ so we can write a stored procedure that uses ; inside it.

2. Create procedure P2(...)

You're creating a procedure named P2 that takes:

rno1: the student's roll number

name1: the name of the book

3. Declare i_date date;

Creates a local variable to store the book’s issue date.

4. Declare diff int;

Creates a variable to store the difference in days between issue date and today.

5.

sql

Copy

Edit

select DateofIssue into i_date

from stud

where Rno = rno1 and NameofBook = name1;

This line fetches the issue date of the book for the given student and stores it in i_date.

6.

sql

Copy

Edit

SELECT DATEDIFF(CURDATE(), i_date) into diff;

Calculates how many days have passed since the book was issued and stores it in diff.

7.

sql

Copy

Edit

If diff > 15 then

  Update stud

  set status = 'R'

  where Rno = rno1 and NameofBook = name1;

End if;

If the book is more than 15 days overdue, it updates the status of that book to 'R' in the stud table.

'R' might stand for "Return" or "Reminder", depending on your system design.

8. delimiter ;

Restores the normal semicolon (;) delimiter.

9. call p2(1, 'DBMS');

Calls the procedure with:

Roll number = 1

Stored Procedure: Purpose

This procedure:

Checks if a book has been issued 15 to 30 days ago.

If yes:

Calculates fine: ₹5 per day.

Stores the fine info in a fine table.

Updates the student’s status to 'R'.

```
delimiter $

Create procedure P3(
   In rno1 int(3),
   name1 varchar(30)
)
begin
   Declare i_date date;
   Declare diff int;
   Declare fine_amt int;

   select Idate into i_date
   from stud
   where rno = rno1 and name = name1;

   SELECT DATEDIFF(CURDATE(), i_date) into diff;

   If (diff >= 15 and diff <= 30) then
      SET fine_amt = diff * 5;

      insert into fine values(rno1, CURDATE(), fine_amt);

      Update stud
      set status = 'R'
      where rno = rno1 and name = name1;
   End if;
end
$

delimiter ;

call p3(1, 'DBMS');
```

```
delimiter $

Create procedure P3(
   In rno1 int(3),
   name1 varchar(30)
)
begin
   Declare i_date date;
   Declare diff int;
   Declare fine_amt int;

   select ldate into i_date
   from stud
   where rno = rno1 and name = name1;

   SELECT DATEDIFF(CURDATE(), i_date) into diff;

   If (diff >= 15 and diff <= 30) then
      SET fine_amt = diff * 5;
      insert into fine values(rno1, CURDATE(), fine_amt);

   elseif (diff > 30) then
      SET fine_amt = diff * 50;
      insert into fine values(rno1, CURDATE(), fine_amt);

   End if;

   Update stud
   set status = 'R'
   where rno = rno1 and name = name1;
End;

$

delimiter ;
```